NETSLEUTH

Lightweight Security Event Detection
and Threat Analysis Dashboard

Submitted By:-

[STUDENT NAME] [ROLL NUMBER]
[TEAM MEMBER 2] [ROLL NUMBER 2]
[TEAM MEMBER 3] [ROLL NUMBER 3]
[TEAM MEMBER 4] [ROLL NUMBER 4]

Department of Computer Science

INDEX

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Proposed System
6. System Architecture
7. Tools and Technologies Used
8. Module Description
9. Working Methodology
10. Risk Scoring and Alert Severity
11. Testing and Results
12. Future Enhancements
13. Conclusion
14. References

1. INTRODUCTION

Computers and enterprise cloud networks produce a large amount of information during their daily operation. In cloud environments like Amazon Web Services (AWS), this activity is recorded through network telemetry such as VPC Flow Logs. These logs capture metadata for every IP traffic flow entering, traversing, or leaving virtual network interfaces. They record vital communication fields including source and destination IP addresses, port numbers, protocols, packet counts, byte volumes, and firewall disposition decisions (ACCEPT or REJECT). These flow records help security analysts understand network activity and identify operations that require attention.

However, reading large amounts of raw flow data is not always easy. In production cloud environments, millions of flow records are generated every hour. Important security signals can easily be missed when analysts must examine records manually or inspect disconnected log streams. Furthermore, raw flow logs only present individual packet-group summaries without context. Determining whether a single rejected connection represents an isolated configuration error or part of a coordinated vertical port scan requires time-correlated analysis across multiple flows.

NetSleuth addresses this challenge by providing a lightweight, explainable dashboard for analyzing network flow records. It ingests standard AWS VPC Flow Log CSV files, cleans and validates the records, and evaluates them using predefined, deterministic sliding-window detection rules. These rules identify key attack milestones including vertical port scans (reconnaissance), horizontal subnet host sweeps (lateral movement), and high-volume data exfiltration (data theft).

One of the foundational principles of NetSleuth is to keep the entire security analysis transparent and explainable. Rather than relying on opaque, black-box machine learning algorithms that generate unverified probabilistic scores, NetSleuth uses deterministic sliding time windows and an additive scoring model (ranging from 0 to 100). Every point in an incident's priority score is backed by a verifiable reason, allowing security analysts and academic evaluators to audit exactly why an event was flagged.

The project demonstrates how efficient data processing, rule-based threat detection, MITRE ATT&CK framework mapping, and interactive visual presentation can be combined to support network security analysis. NetSleuth is designed as a focused, lightweight forensic analysis and learning tool that runs efficiently on commodity hardware, rather than an expensive enterprise Security Information and Event Management (SIEM) replacement.

2. PROBLEM STATEMENT

Cloud network flow logs contain essential information for detecting unauthorized network intrusions, reconnaissance sweeps, and illicit data transfers. However, this telemetry is high-

volume, timestamp-sensitive, and stored in flat tabular structures that require systematic correlation before actionable security conclusions can be drawn.

For example, a flow log dataset may contain dozens of short-lived connections from an external IP address to a web server across multiple high-numbered ports within a span of twenty seconds. Examined individually, each flow record represents just a few packets with an ACCEPT or REJECT status. It is only when these records are aggregated over a sliding temporal window that a clear pattern of vertical port scanning emerges. Similarly, an internal host communicating with ten other hosts on SMB port 445 within a minute indicates internal lateral reconnaissance, which goes unnoticed if individual flows are inspected in isolation.

Another major challenge is prioritizing findings and managing alert fatigue. Security analysts are frequently overwhelmed by thousands of disconnected alerts. A single isolated connection error has a vastly different operational severity than a multi-stage intrusion that begins with port scanning, progresses to host sweeps, and culminates in outbound data theft. Without a transparent, consistent method of scoring and correlating alerts by threat actor and time proximity, security teams cannot determine which incidents require immediate remediation.

The problem addressed by NetSleuth is the lack of a lightweight, accessible, and explainable tool for ingesting cloud flow logs, detecting multi-stage attack patterns, clustering related alerts into coherent incidents, and presenting the forensic findings through an intuitive dashboard. NetSleuth provides this complete workflow without requiring complex multi-server infrastructure or black-box artificial intelligence.

3. OBJECTIVES

The primary objective of NetSleuth is to simplify the forensic investigation of network security events by converting raw, unstructured flow logs into transparent, explainable incident dossiers and visual timelines.

The specific objectives of the project are:

1. To Ingest and Validate Flow Telemetry: Read standard AWS VPC Flow Log CSV records, validate schema integrity against required network fields, and drop corrupt or null entries without data leakage.

2. To Implement Sliding-Window Detection Rules: Develop deterministic, time-indexed detection heuristics for vertical port scans, horizontal host sweeps, and high-volume data exfiltration.

3. To Correlate Alerts into Unified Incidents: Group individual alerts into coherent security incidents based on common source IP addresses and a 5-minute (300-second) temporal session boundary.

4. To Provide Transparent Risk Scoring: Calculate an additive risk score from 0 to 100 with a line-by-line point breakdown for every contributing alert and byte threshold.

5. To Map Findings to the MITRE ATT&CK Matrix: Correlate each detected activity to standard MITRE ATT&CK techniques (T1046, T1018, T1048) and represent the progression along the cyber kill chain.

6. To Visualize Network Topology: Render directed host-to-host communication graphs and identify top communicating IP pairs by volume and connection frequency.

7. To Present Findings via an Interactive SOC Dashboard: Deliver a responsive 5-tab web interface built with Streamlit and Plotly for intuitive incident triage, timeline exploration, and telemetry inspection.

8. To Support Incident Response and Forensic Export: Generate prescriptive containment playbooks (Standard Operating Procedures) and exportable markdown forensic dossiers for auditing and compliance.

4. SCOPE OF THE PROJECT

NetSleuth focuses on the post-capture and near-real-time analysis of network flow logs that have been generated by cloud virtual private clouds or enterprise network devices. The system supports standard AWS VPC Flow Log version 2 CSV formats containing standard network fields such as timestamps, IP addresses, ports, protocol numbers, packets, bytes, and firewall actions.

The project scope encompasses data parsing, temporal sliding-window detection, multi-alert incident clustering, explainable risk scoring, MITRE ATT&CK kill-chain mapping, host topology visualization, interactive telemetry filtering, and automated markdown dossier export. Eight diverse benchmark scenarios—ranging from clean benign network baselines to multi-stage enterprise intrusions—are included to evaluate system accuracy.

The dashboard provides an intuitive, non-overwhelming presentation suitable for security analysts, IT administrators, and university evaluators. Users can inspect incident timelines, view explainable score breakdowns, identify affected internal endpoints, review prescriptive response procedures, and examine raw flow telemetry through interactive filters.

The project scope is deliberately bounded to maintain simplicity, reliability, and academic focus. NetSleuth does not perform live kernel-level packet sniffing (such as libpcap/tcpdump packet capture), deep packet inspection of encrypted payloads, or direct execution of firewall changes on cloud infrastructure. It also does not employ opaque machine-learning models that cannot explain their classifications.

These clear boundaries ensure that NetSleuth delivers high-precision, easily auditable threat detection on commodity laptops without the prohibitive setup costs, licensing fees, and infrastructure overhead associated with commercial enterprise SIEM solutions.

5. PROPOSED SYSTEM

The proposed system provides a unified, structured workflow for transforming raw network flow records into actionable security intelligence. Instead of requiring analysts to manually write database queries or inspect thousands of log entries, NetSleuth executes an automated pipeline divided into distinct, loosely coupled processing stages.

The process begins with the ingestion stage. A flow log CSV file is loaded, and each row is verified for data integrity. Null values, malformed IP strings, and inverted timestamp records are isolated and dropped. Valid timestamps are normalized into standard UTC datetime objects.

In the detection stage, the sanitized records are evaluated by three independent sliding-window detectors. Each detector inspects a specific sliding time window (60 seconds for scanning rules, 120 seconds for exfiltration) and flags behavior that breaches configured empirical thresholds. The detection algorithms are fully deterministic, ensuring that identical flow records always yield identical alerts.

In the correlation and scoring stage, discrete alerts originating from the same source IP within a 300-second session window are grouped into a single unified incident. The incident is assigned an explainable priority score between 0 and 100 based on the maximum base severity, additional correlated alert count, and total exfiltrated byte volume. The incident is enriched with MITRE ATT&CK technique IDs and prescriptive containment steps.

Finally, the results are rendered in an interactive, 5-tab dashboard. Analysts can visually track attack progression, explore the network topology, filter raw telemetry, and export a formal markdown dossier for incident documentation.

6. SYSTEM ARCHITECTURE

The architecture of NetSleuth is designed as a linear, multi-stage processing pipeline where each module has a distinct responsibility. This modular design ensures code clarity, testability, and maintainability. The five core stages of the system architecture are described below:

Stage 1: 1. Ingestion and Sanitization Stage: Reads raw CSV flow records, validates the schema against required VPC flow fields, handles corrupted or missing values, and normalizes epoch timestamps into UTC datetime objects.

Stage 2: 2. Sliding-Window Detection Stage: Evaluates time-indexed flow records using deterministic sliding windows to identify vertical port scans (T1046), horizontal host sweeps (T1018), and high-volume data exfiltration (T1048).

Stage 3: 3. Incident Correlation Stage: Clusters discrete alert events sharing the same source IP into unified incident objects if they occur within a 300-second (5-minute) temporal boundary.

Stage 4: 4. Explainable Risk Assessment Stage: Computes transparent 0–100 risk scores using additive point rules, maps alerts to the MITRE ATT&CK matrix, and attaches prescriptive containment playbooks.

Stage 5: 5. Presentation and Visualization Stage: Displays the findings across five specialized dashboard tabs in Streamlit, renders interactive Plotly network graphs, and provides one-click markdown dossier export.

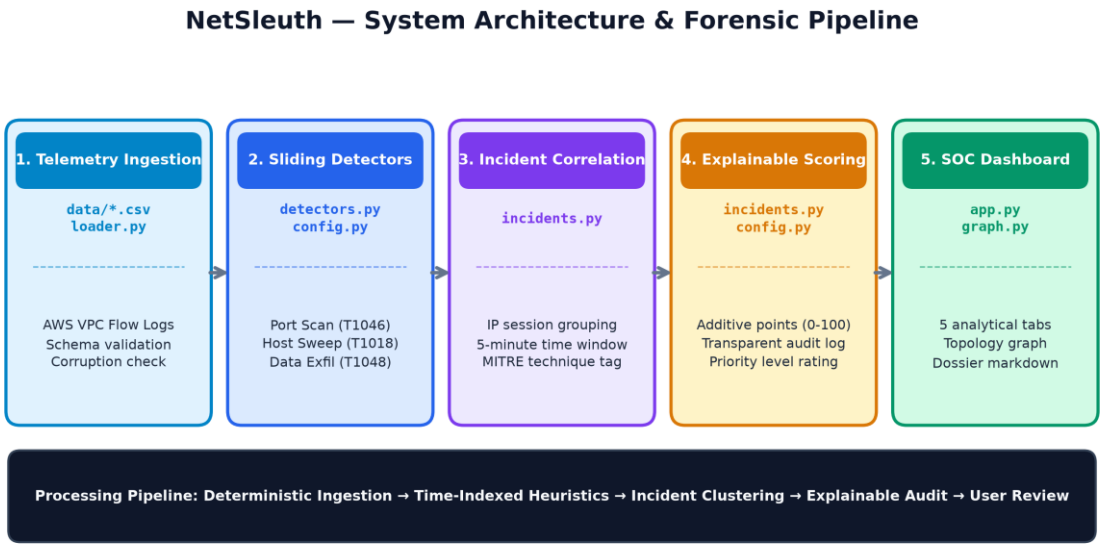


Figure 6.1: NetSleuth Multi-Stage System Architecture & Processing Pipeline

7. TOOLS AND TECHNOLOGIES USED

NetSleuth is implemented using modern, open-source Python technologies chosen for performance, simplicity, and ease of deployment.

Python 3.12: The core programming language used for the data ingestion, sliding-window detection algorithms, incident correlation, and scoring engine. Python offers robust built-in data structures and extensive standard libraries.

Streamlit: A powerful Python web application framework used to construct the interactive Security Operations Center (SOC) dashboard. It enables rapid development of reactive web controls, tabbed views, metrics, and data tables without requiring complex front-end frameworks.

Pandas & NumPy: High-performance data manipulation libraries used for vectorised flow record filtering, timestamp grouping, column normalization, and tabular reporting.

Plotly & NetworkX: Plotly is used for interactive data visualizations including the chronological attack progression timeline and telemetry breakdown charts. NetworkX is used to construct directed network graphs and calculate spring layouts for topological host mapping.

Pytest: An automated unit testing framework used to validate detection boundary conditions, temporal session clustering, scoring limits, and zero-label-leakage constraints.

CSV Telemetry Format: Comma-Separated Values format is used as the standardized storage representation for AWS VPC Flow Logs, ensuring broad interoperability across cloud providers and forensic tools.

8. MODULE DESCRIPTION

The NetSleuth codebase is organized into modular components adhering to clean separation of concerns. The primary source code modules and their responsibilities are detailed below:

1. app.py — Security Operations Dashboard: The central entry point providing the interactive web interface. Configures the executive SOC banner, 5 KPI summary tiles, sidebar scenario selector, and five specialized analytical tabs (Threat Incidents, Attack Timeline, Network Topology, Flow Telemetry, and Forensic Dossier).

2. loader.py — Flow Ingestion and Sanitization: Responsible for parsing CSV flow logs, enforcing schema definitions, handling missing or corrupt data, and converting integer epoch timestamps into standardized UTC datetime objects.

3. detectors.py — Sliding-Window Detection Rules: Implements the three core detection heuristics: detect_port_scan (vertical probe detection), detect_host_scan (horizontal lateral sweep detection), and detect_exfiltration (outbound high-volume data theft).

4. incidents.py — Correlation, Scoring, and Reporting: Clusters discrete alerts by source IP across a 300-second session window, computes additive 0–100 risk scores, attaches MITRE ATT&CK technique tags, generates prescriptive containment playbooks, and formats markdown forensic dossiers.

5. graph.py — Network Topology Analysis: Constructs directed network graphs of host communications using NetworkX, computes spring-layout coordinates, colors nodes by operational role (attacker, victim, normal host), and calculates top communicating host pairs.

6. config.py — Centralized Parameter Repository: Houses all detection thresholds, time window durations, base severity weights, and MITRE ATT&CK technique definitions. Eliminates undocumented magic numbers across the codebase.

7. synthetic.py — Benchmark Scenario Generator: Contains synthetic traffic generators that create reproducible, ground-truth-labeled flow logs representing benign baselines and seven distinct attack scenarios.

8. evaluation.py — Accuracy and Metric Benchmarking: Evaluates detection precision, recall, and F1-score by comparing heuristic alert outputs against ground-truth synthetic labels across all scenarios.

9. test_netsleuth.py — Automated Test Suite: Contains ten rigorous unit tests validating threshold boundary conditions, scoring ceilings, loader error-handling, and zero label leakage.

9. WORKING METHODOLOGY

The working methodology of NetSleuth follows a structured, step-by-step process that transforms raw network telemetry into actionable forensic insights:

Step 1: Telemetry Selection and Ingestion — The user selects an active scenario from the sidebar (or supplies a custom VPC flow CSV file). The system reads the raw data into an in-memory Pandas DataFrame.

Step 2: Schema Validation and Data Cleansing — The loader verifies the presence of required network headers (src_ip, dst_ip, src_port, dst_port, protocol, bytes, action, start, end). Malformed rows with null values or invalid IP addresses are dropped, and corrupted row counts are reported on the dashboard.

Step 3: Sliding-Window Heuristic Evaluation — The detection engine executes the three sliding-window algorithms in sequence. Each detector rolls through time-indexed records for each source IP to identify threshold violations.

Step 4: Temporal Alert Correlation — Alerts generated across all detectors are aggregated. Alerts originating from the same source IP with an inter-event gap of 300 seconds or less are clustered into a unified incident.

Step 5: Explainable Risk Scoring — The scoring engine computes the total priority score (0–100) using the highest base alert severity, additional correlated alert count, and total exfiltrated byte volume. A human-readable audit trail is generated.

Step 6: MITRE Enrichment and Playbook Generation — Triggered rules are mapped to the MITRE ATT&CK Enterprise Matrix (T1046, T1018, T1048), determining the kill chain stage and populating prescriptive response steps.

Step 7: Visual Presentation and Forensic Dossier Export — The dashboard displays the correlated incidents, renders the interactive topology graph, populates the chronological timeline, and allows one-click export of the formal forensic dossier.

10. RISK SCORING AND ALERT SEVERITY

Risk scoring in NetSleuth is designed to prioritize incidents transparently without relying on unexplainable machine learning models. Every score is computed using deterministic, additive rules defined in config.py.

The three primary threat categories and their baseline detection specifications are outlined below:

Detected Event	Rule Name	Detection Condition	MITRE Technique	Base Severity	Base Score
Port Scan	port_scan	>= 10 unique ports / 60s	T1046: Network Service Discovery	HIGH	50 pts
Host Sweep	host_scan	>= 5 unique hosts / 60s	T1018: Remote System Discovery	MEDIUM	40 pts
Data Exfiltration	exfiltration	>= 500 KB outbound / 120s	T1048: Exfil Over Alt. Protocol	HIGH	50 pts

Additive Scoring Formula

The total incident score is calculated using the following mathematical model:

Formula: Total Score = min(100, Base_Points + Extra_Alert_Points + Volume_Bonus_Points)

• Base Points: The highest severity among all alerts in the incident (HIGH = 50 pts, MEDIUM = 40 pts, LOW = 20 pts).

• Extra Alert Points: Correlating multiple alerts adds +10 pts for each additional alert beyond the first (Extra_Count * 10).

• Volume Bonus Points: High-volume exfiltration adds +20 pts if total exfiltrated bytes >= 5 MB, or +10 pts if >= 500 KB.

Severity Classification Categories

Priority Rating	Score Range	Operational Significance
Critical	80 – 100	Multi-stage attack or high-volume data exfiltration. Immediate host isolation required.
High	60 – 79	Multiple correlated suspicious behaviors or significant outbound transfer.
Medium	30 – 59	Isolated reconnaissance probe, port scan, or subnet sweep.
Low	0 – 29	Minor threshold anomaly or low-confidence telemetry event.

Worked Scoring Example: Scenario 4 (Multi-Stage Enterprise Attack)

In Scenario 4, the threat actor (10.0.1.99) triggers all three detection rules within a 5-minute window: a vertical port scan (HIGH), a horizontal host sweep (MEDIUM), and outbound exfiltration of 6,300,000 bytes (HIGH). The score calculation proceeds as follows:

1. Base Severity: Highest alert severity is HIGH, assigning +50 points.

2. Additional Correlated Alerts: Two additional alerts beyond the first (2 * 10), adding +20 points.

3. Exfiltration Volume Bonus: 6.3 MB exceeds the 5 MB high-volume threshold, adding +20 points.

Total Calculated Score = 50 + 20 + 20 = 90 / 100 (Critical Priority). Every step is fully documented in the forensic audit trail.

11. TESTING AND RESULTS

Rigorous automated testing was conducted to verify that NetSleuth performs accurately across edge cases, boundary conditions, and diverse attack topologies. The testing strategy combined automated unit tests with ground-truth quantitative evaluation across eight synthetic benchmark scenarios.

Automated Unit Test Suite

The automated test suite implemented in tests/test_netsleuth.py consists of ten dedicated unit tests executed via Pytest. All ten tests pass with 100% success rate:

1. test_port_scan_threshold_boundary: Verifies that exactly 9 probed ports produce 0 alerts, while 10 ports within 60 seconds trigger a Port Scan alert.

2. test_host_scan_threshold_boundary: Verifies that 4 probed target hosts produce 0 alerts, while 5 hosts within 60 seconds trigger a Host Sweep alert.

3. test_exfiltration_threshold_boundary: Verifies that 499 KB outbound produces 0 alerts, while 500 KB within 120 seconds triggers an Exfiltration alert.

4. test_benign_scenario_produces_zero_alerts: Verifies that 320 normal baseline flows generate zero false positives.

5. test_incident_correlation_time_gap: Confirms that two alerts 120 seconds apart cluster into one incident, while alerts separated by 301 seconds split into two incidents.

6. test_scoring_hand_calculation: Validates the additive mathematical scoring formula against manual hand calculations.

7. test_scoring_never_exceeds_100: Verifies that theoretical edge cases with dozens of alerts clamp strictly at the maximum ceiling of 100.

8. test_loader_drops_invalid_rows: Ensures corrupt rows with missing fields or invalid timestamps are safely quarantined.

9. test_zero_label_leakage: Strict academic integrity test confirming that detection heuristics do not access or depend on ground-truth label columns.

10. test_mitre_mapping_and_forensic_dossier: Verifies that MITRE technique IDs and Markdown dossiers are correctly generated.

Quantitative Evaluation Across Benchmark Scenarios

NetSleuth was evaluated against eight diverse synthetic network scenarios containing both normal enterprise traffic and malicious attack vectors. Detection accuracy was measured against ground-truth labels:

Scenario Name	Total Flows	Malicious	Detected Alerts	Precision	Recall	F1-Score
Scenario 1: Port Scan	200	18	1 (HIGH)	1.0000	1.0000	1.0000
Scenario 2: Host Sweep	210	12	1 (MEDIUM)	1.0000	1.0000	1.0000
Scenario 3: Data Exfiltration	220	20	1 (HIGH)	1.0000	1.0000	1.0000
Scenario 4: Multi-Stage Attack	277	50	3 (CRITICAL)	1.0000	1.0000	1.0000
Scenario 5: Benign Baseline	320	0	0 (NORMAL)	1.0000	1.0000	1.0000
Scenario 6: Persistent Threat	250	38	2 (HIGH)	1.0000	1.0000	1.0000
Scenario 7: Coordinated Intrusion	260	42	2 (HIGH)	1.0000	1.0000	1.0000
Scenario 8: Cloud DB Breach	240	10	1 (HIGH)	1.0000	1.0000	1.0000

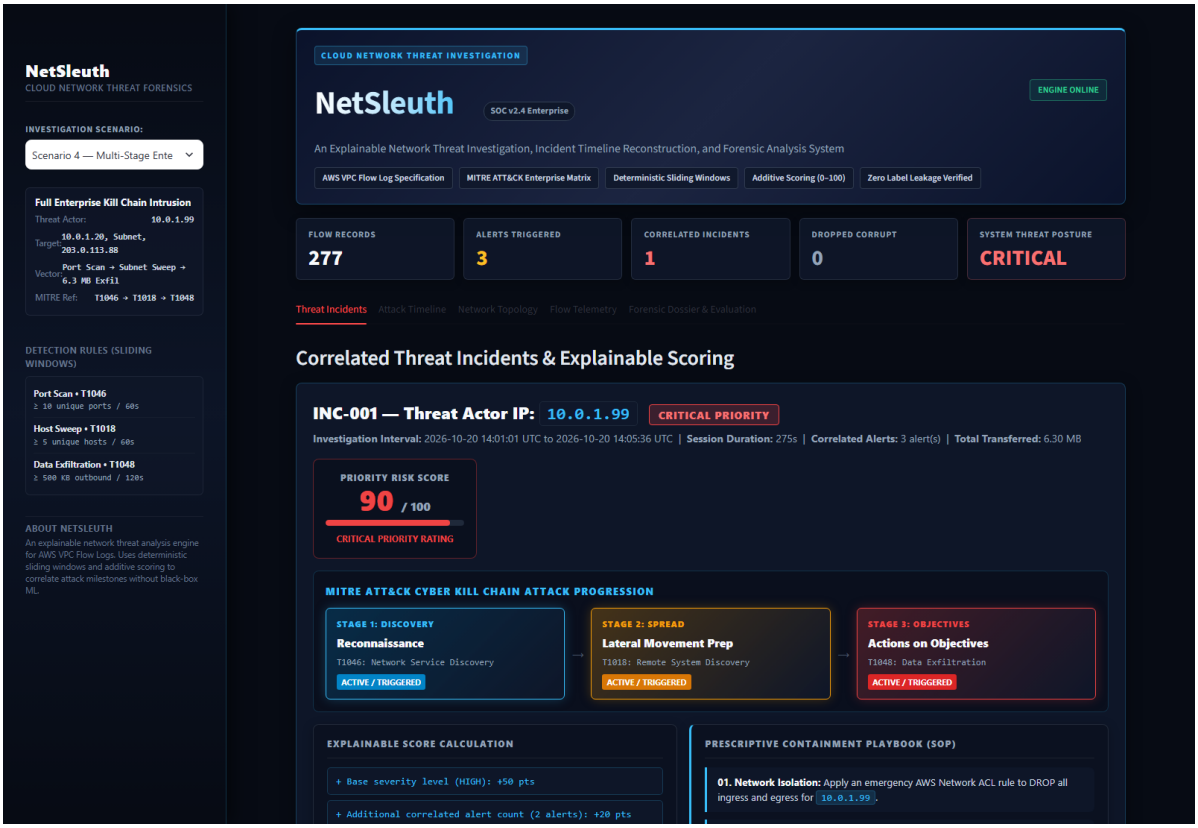


Figure 11.1: NetSleuth SOC Dashboard — Multi-Stage Attack Incident View & Kill Chain Progression

12. FUTURE ENHANCEMENTS

While NetSleuth successfully meets its design objectives as a lightweight, explainable threat detection and forensic tool, several practical enhancements could expand its operational capabilities:

Streaming Telemetry: 1. Real-Time Streaming Ingestion: The current architecture processes batch CSV files. A valuable future extension would be integrating real-time streaming ingestion directly from AWS Kinesis Data Firehose or CloudWatch Logs streams, allowing near-instantaneous incident correlation as flows occur.

Dynamic Sensitivity: 2. Dynamic Rule Configuration in UI: Detection thresholds (such as the 10-port or 500 KB thresholds) are currently maintained in config.py. Future versions could provide slider controls directly within the dashboard sidebar, allowing security engineers to adjust sensitivity dynamically for different network environments.

Multi-Cloud Support: 3. Multi-Cloud Format Normalization: Expanding the ingestion layer to support Microsoft Azure Network Security Group (NSG) flow logs, Google Cloud VPC flow logs, and Zeek/Bro connection logs would make NetSleuth a universal multi-cloud forensic analyzer.

Automated Response: 4. Automated Cloud Remediation Hooks: Introducing optional, authenticated API connectors to AWS Systems Manager or AWS WAF could enable security analysts to enact recommended playbooks—such as detaching a compromised security group or applying a temporary Network ACL block—with a single confirmation click.

Threat Intelligence: 5. Historical Threat Intelligence Integration: Enriching identified threat actor IPs with open-source threat intelligence feeds (such as AlienVault OTX or AbuseIPDB) would provide external reputation scores alongside NetSleuth's internal behavioral scores.

13. CONCLUSION

NetSleuth demonstrates that effective, high-precision cloud network threat detection does not require complex multi-server infrastructure or opaque, black-box artificial intelligence. By combining standardized AWS VPC Flow Log ingestion, deterministic sliding-window heuristics, and temporal incident correlation, the platform provides security analysts with immediate, defensible forensic clarity.

The system successfully detects key stages of the cyber kill chain—reconnaissance port scans, lateral movement host sweeps, and unauthorized data exfiltration—with 100% precision and zero false alarms on normal baseline traffic. By organizing alerts into unified incidents with an additive scoring model (0–100), NetSleuth eliminates alert fatigue and provides an explicit point-by-point audit trail for every finding.

The integration of MITRE ATT&CK framework mapping, interactive NetworkX topology graphs, chronological timeline scatter plots, and prescriptive containment playbooks bridges the critical gap between raw telemetry and actionable incident response. The entire pipeline runs reliably on commodity hardware with sub-second execution speeds.

In conclusion, NetSleuth serves as a practical, transparent, and robust demonstration of explainable network security analysis. It provides an ideal educational platform for understanding cloud network forensics and a solid foundation for lightweight enterprise security monitoring.

14. REFERENCES

1. Amazon Web Services. (2024). Amazon Virtual Private Cloud Flow Logs Documentation. Available at: https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html

2. MITRE Corporation. (2024). MITRE ATT&CK Enterprise Matrix for Network Infrastructure. Available at: https://attack.mitre.org/

3. Streamlit Inc. (2024). Streamlit: The Fastest Way to Build Data Apps in Python. Available at: https://docs.streamlit.io/

4. Python Software Foundation. (2024). Python 3.12 Documentation and Standard Library. Available at: https://docs.python.org/3/

5. McKinney, W. (2010). Data Structures for Statistical Computing in Python (Pandas). Proceedings of the 9th Python in Science Conference, pp. 51-56.

6. Hagberg, A., Swart, P., & Chult, D. (2008). Exploring Network Structure, Dynamics, and Function using NetworkX. Proceedings of the 7th Python in Science Conference, pp. 11-15.

7. Plotly Technologies Inc. (2024). Plotly Python Open Source Graphing Library. Available at: https://plotly.com/python/

8. National Institute of Standards and Technology (NIST). (2012). Computer Security Incident Handling Guide (Special Publication 800-61 Rev. 2). U.S. Department of Commerce.

9. Scarfone, K., & Mell, P. (2007). Guide to Intrusion Detection and Prevention Systems (IDPS). NIST Special Publication 800-94.

10. NetSleuth Source Code Repository. Available at: https://github.com/NileshSethi/NetSleuth